An Efficient and Precision-Reconfigurable Digital CIM Macro for DNN Accelerators

Dingyang Zou, Gaoche Zhang, Xu Zhang, Meiqi Wang, and Zhongfeng Wang

***Abstract*— Due to the demand for high energy efficiency in deep neural network (DNN) accelerators, computing-in-memory (CIM) is becoming increasingly popular in recent years. However, current CIM designs suffer from high latency and insufficient flexibility. To address the issues, this brief proposes a Booth-multiplication-based CIM macro (BCIM) with modified Booth encoding and partial product (PP) generation method specially designed for CIM architecture. In addition, a methodology is presented for designing precision-reconfigurable digital CIM macros. We also optimize the precision-reconfigurable shift adder in the macro based on the cutting down carry connection method. The design attains a performance of 2048 GOPS and a peak energy efficiency of 79.15 TOPS/W in the signed INT4 mode at a frequency of 500 MHz.**

***Index Terms*— Computing-in-memory (CIM), multiply–accumulate (MAC), precision-reconfigurable, static random access memory (SRAM).**

I. Introduction

In deep neural network (DNN) models, there are a large number of multiply–accumulate (MAC) operations that result in substantial data movement and computation. However, in the traditional Von Neumann architecture, computation and storage are separated, resulting in energy overhead and high delay costs. Computing-in-memory (CIM), which performs the computation in memory and avoids most data movement costs, has become increasingly popular [1], [2].

Compared with analog CIM macros, digital CIM macros generate partial products (PPs) in the memory array and use the adder tree and shift adder to accumulate them, offering high precision and accuracy, which are usually limited in analog CIM macros [3]. As a result, digital CIM macros are more widely used in DNN models that require higher precision in training and inference.

Previous works on digital CIM macros have demonstrated significant energy efficiency and throughput advantages over conventional architectures. However, several issues still exist in these works.

1) *Issue 1:* Shift adders are commonly used to accumulate the sum of PPs to achieve high energy efficiency in digital CIM macros. The majority of prior research have used a conventional approach for generating PPs [4], [5], [6], [7]. However, these methods have not been optimized to minimize the number of PPs, which in turn increases the cycles required for the shift-add operation, consequently degrading the throughput and energy efficiency.
2) *Issue 2:* Supporting multiprecision signed matrix multiplication is crucial for DNN accelerators. However, designing precision-reconfigurable CIM macros for signed matrix multiplication is hard because of the existence of sign extensions. Sign extension results in significant differences in multiplication and addition between signed and unsigned numbers. Therefore, the traditional method [7], [8], [9] for designing precision-reconfigurable hardware becomes ineffective, as shown in Fig. 1.

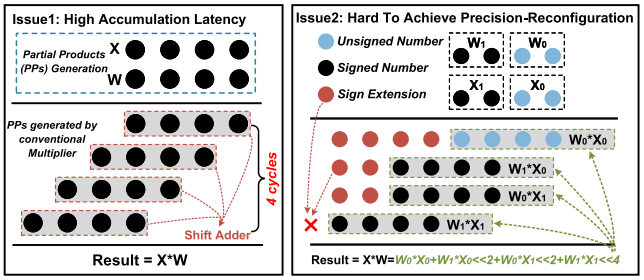


Fig. 1. Issues in previous works.

Furthermore, certain prior studies have proposed precision-reconfigurable CIM macros for signed matrix multiplication. However, these macros have not been thoroughly optimized. The precision-reconfigurable design in [11] has to disable a local MAC array in INT8 mode, causing low hardware utilization. Colonnade [10] changed the number representation of inputs, but the computation circuit is more complex, causing area and energy overhead. Tu et al. [5] proposed a macro efficiently supporting both the signed INT and floating point (FP) computing, but this method has to add a circuit to control all the sign extension bits, and adders with large bit-width are used to merge the outputs of the 8-bit mode to generate outputs of the 16-bit mode and 24-bit mode. Moreover, these works did not discuss whether their methods can be extended to support a wider range of precisions.

To address the above issues, we design a Booth-multiplication-based digital precision-reconfigurable SRAM-CIM macro for DNN accelerators, along with a methodology for designing precision-reconfigurable digital CIM macros for signed MAC. We summarize our contributions as follows.

1) For Issue 1, we propose the Booth-multiplication-based CIM macro (BCIM) with modified Booth encoding and PP generation method for CIM architecture, thus reducing the number of PPs by half and consequently reducing accumulation cycles by half. Compared with previous works, we better combine the characteristics of the architecture of digital CIM macro and the PP generation method of Booth multiplication.
2) For Issue 2, we propose the methodology for designing a precision-reconfigurable digital CIM macro by redesigning the

Manuscript received 9 March 2024; revised 4 August 2024; accepted 14 August 2024. Date of publication 24 September 2024; date of current version 23 January 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4400600 and in part by the National Natural Science Foundation of China under Grant 62404256. *(Corresponding authors: Zhongfeng Wang; Meiqi Wang.)*

Dingyang Zou, Gaoche Zhang, and Xu Zhang are with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China (e-mail: dyzou@smail.nju.edu.cn; gczhang@smail.nju.edu.cn; zhangxu96@smail.nju.edu.cn).

Meiqi Wang is with the School of Integrated Circuits, Sun Yat-sen University, Shenzhen 518107, China (e-mail: wangmq53@mail.sysu.edu.cn).

Zhongfeng Wang is with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China, and also with the School of Integrated Circuits, Sun Yat-sen University, Shenzhen 518107, China (e-mail: zfwang@nju.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2024.3455091

1063-8210 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TRUTH TABLE OF THE PROPOSED BOOTH ENCODING METHOD

Input			Encoding Signals			Partial Product		
$I[2i+1]$	$I[2i]$	$I[2i-1]$	Two	Neg	Zero	PPi	PPi[0]	Comp
0	0	0	X	X	1	X	X	0
0	0	1	0	0	0	$\overline{W}$	$\overline{W}[0]$	0
0	1	0	0	0	0	$\overline{W}$	$\overline{W}[0]$	0
0	1	1	1	0	0	$\overline{2W}$	1	0
1	0	0	1	1	0	$2W$	0	1
1	0	1	0	1	0	W	$W[0]$	1
1	1	0	0	1	0	W	$W[0]$	1
1	1	1	X	X	1	X	X	0

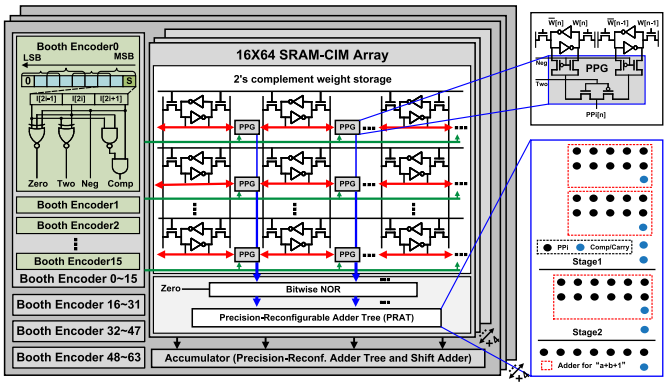


Fig. 2. Proposed CIM macro architecture.

carry connections of the sign extension bits of adders and skipping and gating the useless bits of adders in the subadder trees (subtrees). Precision-reconfigurable macros designed using the proposed methodology have the following three advantages: 1) high hardware utilization; 2) supporting precision reconfiguration with little additional circuit; and 3) reconfigurable for a wide range of precisions.

3) Besides, we propose the precision-reconfigurable shift adder (PRSA) based on the insight that an adder with large bit-width can be divided into several adders with small bit-width by cutting down the carry connection. The proposed shift adder achieves high energy efficiency by fully using each bit of the shift adder under different precision modes.

II. ARCHITECTURE AND METHODOLOGY

A. CIM Macro Architecture

In this section, the Booth-multiplication-based digital precision-reconfigurable SRAM-CIM macro is proposed. Fig. 2 shows the architecture of the proposed CIM macro.

The proposed design consists of four 6T bit-cell SRAM arrays with a size of 16 × 64, SRAM read/write peripherals, address decoder drivers, Booth PP generators (PPGs), adder trees, and shift adders. The inputs of PPGs are extracted from the cell, and the PPs are propagated to the adder tree in a parallel manner as in [5], [6], and [8]. Large arrays can be implemented by stacking multiple arrays shown in Fig. 2. We stack four subarrays to implement an SRAM-CIM macro with the SRAM capacity of 16 kb.

The methodology can support signed INT$\{1 \cdot P, 2 \cdot P, \ldots, n \cdot P\}$. For example, when we set $P = 4$ and choose to support the precision of signed INT4 and INT8, the macro performs parallel MAC operations, computing signed INT4/INT8 MAC per two/four cycles. In the INT4 mode, each row of the SRAM array stores a 4-bit weight in every four columns. In the INT8 mode, each row stores an 8-bit weight in every eight columns, which means that every two 4-bit weights combine to form an 8-bit weight. The macros supporting other precisions follow the same principle.

B. Booth-Multiplication-Based CIM Macro

The PP generation method of Radix-4 Booth multiplication is most suitable for CIM architecture. Radix-4 Booth multiplication generates four kinds of PPs $\{W, 2W, -W, -2W\}$ (W is the weight stored in the SRAM array) by controlling the multiplicand with consecutive 3 bits of the multiplier. To implement the CIM macro with a PP generation method based on Booth multiplication, our work focuses on two main aspects. One aspect is the modified Booth encoding method for PPs and encoding signals, while the other aspect is the analog PPG circuit.

Inspired by [5], we divide the PPs into two components: the intermediate PP (PPi) and the compensation (Comp). PPi is generated by the analog PPG circuit, while Comp is generated by the digital Booth encoder. PPs can be obtained by performing an addition operation between PPi and Comp. Based on the insight that the adder for "$a + b + 1$" can be designed by changing the half adder in the least significant bit (LSB) of the adder for $a + b$ into a full adder, we merge the addition of PPi and Comp to the adder tree for the accumulation of PPs. As shown in the right of Fig. 2, we take the accumulation of four PPis and Comps as an example, and the addition of three Comps is merged to each stage of the adder tree. The addition of the remaining one Comp is merged into the shift adder with the same method. To support this operation, we let the PPi represent $\{W, 2W, \overline{W}, \overline{2W}\}$ and the Comp be $\{0, 1\}$. In [5], PPi represents $\{W, 2W, \overline{W}, 2\overline{W}\}$ and Comp represents $\{0, 1, 2\}$. For this encoding method, it is impossible to merge the addition of PPi and Comp to the adder tree since Comp can take on the value of 2. In this case, the "$a + b + 1$" method cannot work.

The encoding circuit generates three encoding signals and Comp. The three encoding signals are Zero, Two, and Neg. We optimize the encoding circuit based on the insight that it does not matter what the signal Two or Neg is when Zero $= 1$, as the PP is obtained by performing a bitwise NOR operation between PPi and the Zero signal. Therefore, as shown in Table I, we set the other signals to X while Zero $= 1$, so that the combinational logic of the encoder can be simplified.

For the analog PPG, we discover that the SRAM cell inherently provides both W and $\overline{W}$, and the adjacent 6T cells provide $2W$ and $\overline{2W}$. Therefore, we leverage this feature to design an analog PP generation circuit based on selection. The proposed analog PPG circuit is shown in Fig. 2. We implement it based on transmission gate logic. The selection process occurs in two steps. In the first step, the PPG uses the "Neg" signal to control the four MOSFETs above to determine whether inversion is required or not. In the second step, it uses the "Two" signal to control two MOSFETs below to choose whether to shift left. Low threshold voltage MOSFETs are used in the 6T PPG to alleviate the threshold voltage loss issue caused by cascading pMOS and nMOS in the PPG.

C. Precision-Reconfiguration Methodology

In this section, we propose a methodology for designing a precision-reconfigurable digital CIM macro. Through analysis of the structure of digital CIM macros, we have identified two insights: Insight-(1) the most challenging aspect of designing precision-reconfigurable macros lies in efficiently modifying the structure of the adder tree and Insight-(2) in the adder tree, the bits of adders that use sign extension as their input in one precision mode are useless in another precision mode. For example, in the INT8 mode, an 8-bit PP is divided into two 4-bit PPs and uses two subtrees for accumulation,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:37 UTC from IEEE Xplore. Restrictions apply.

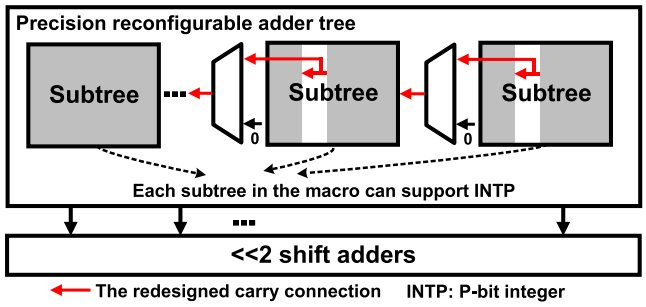


Fig. 3. Architecture of the precision-reconfigurable adder tree designed using the methodology.

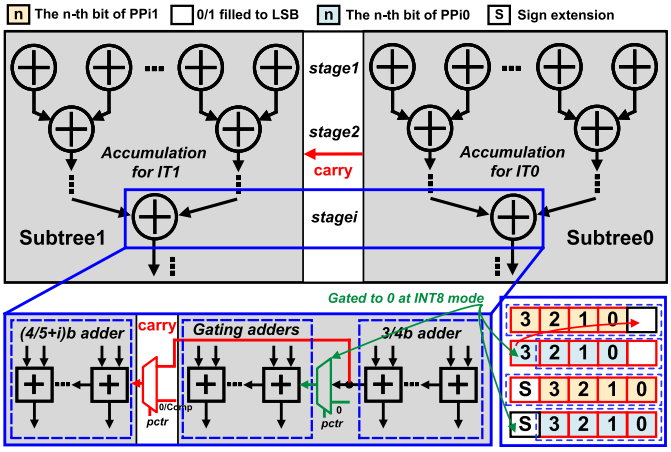


Fig. 4. Connection between two adjacent subtrees for adder trees in traditional macros and Booth macros.

but one PP is an unsigned number, so the sign extension bits in the subtree for the unsigned PP are useless. Considering the two insights, the methodology is designed based on modifying the carry connections of adders between the two adjacent subtrees so that the useless bits of adders can be skipped. Bit-level adder gating is used to reduce energy consumption for the skipped bits of adders. The architecture of the precision-reconfigurable adder tree designed using the methodology is shown in Fig. 3.

The methodology can be used to design macros supporting inputs of arbitrary precisions (the input precision is controlled by the accumulation cycle, as long as the shift adder has a sufficiently large bit-width). For weight precision, it can support signed INT$\{1 \cdot P, 2 \cdot P, \ldots, n \cdot P\}$. Here, P denotes the precision that each subtree in the macro can support. For example, if there is a requirement for the macro to support the weight precision of INT4, INT6, and INT12, we can set the value of P to be 2. The number of subtrees should be 6, which is the least common multiple of 4, 6, and 12 divided by P. This configuration ensures optimal hardware utilization, distinguishing it from the approach described in [11].

Fig. 4 illustrates the details of the connection between the two adjacent subtrees. We set P to be 4 as an example. This example can also be regarded as an adder tree of the macro that supports the weight precisions of signed INT4 and INT8.

The adder in the bottom left corner of Fig. 4 shows the details of the approach to connecting the carries from subtree0 to subtree1. In INT8 mode, the more significant bits of the adders, which are beyond the third LSB (for Booth-based macros, it is fourth, while for traditional macros, it is third), are useless, as their inputs are sign extensions in the INT4 mode. Therefore, these bits of adders can be skipped in the INT8 mode. Controlled by the precision control (pctr) signal, the carry of the fourth significant bit of the adders in subtree0 is connected to the LSB of the adders in subtree1. The PRSA can also be designed using the same "adder skipping" method.

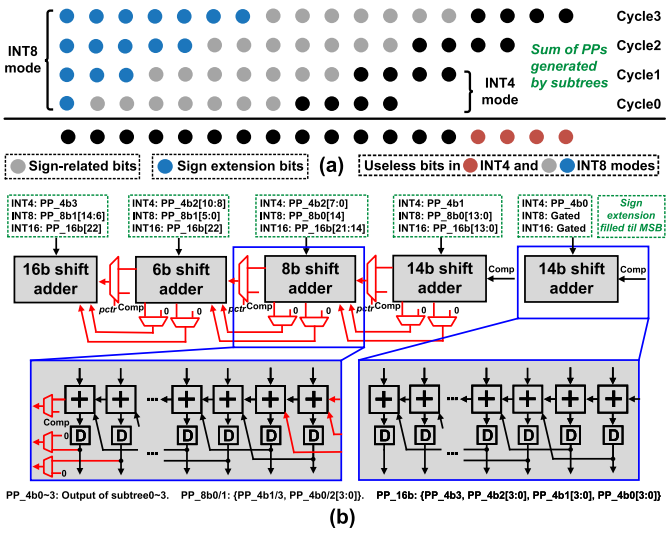


Fig. 5. Precision-reconfigurable shift adder. (a) Low bit utilization of the subshift adder in prior research. (b) Proposed PRSA.

For adder trees in Booth-based macros, we also need to adjust the inputs of the subtrees in different precision modes. In the bottom right corner of Fig. 4 is the approach to adjusting the inputs of the subtrees in different precision modes. For ease of description, we refer to the inputs of subtree0 as IT0 and the inputs of subtree1 as IT1. When using the PP generation method from Booth multiplication, the bit-widths of IT0 and IT1 are 5. In the INT4 mode, the 10 bits of IT0 and IT1 are all valid, while in the INT8 mode, only 9 bits are valid. It is important to discover an efficient approach for choosing the valid 9 bits from the available 10 bits. However, when Two $= 1$ and Two $= 0$, the valid 9 bits are different, leading to a complex hardware implementation for selecting the valid bits. When Two $= 0$, the less significant 4 bits of IT0 are valid, and the 5 bits of IT1 are all valid. When Two $= 1$, the situation is exactly the opposite. To reduce hardware complexity, we choose the less significant 4 bits of IT0 and all the 5 bits of IT1 as the valid 9 bits, no matter what the value of signal Two is. To ensure the correctness of the results, we correct the LSB of IT1 to the MSB of IT0 when Two $= 1$. In the INT8 mode, the invalid bits and the carry to the skipped adders are gated to "0" to reduce dynamic energy consumption for the skipped bits of adders in each stage of the adder tree. The same method can be applied to other precision modes.

D. Precision-Reconfigurable Shift Adder

We have done the bit-level optimization to fully reuse the full adders and DFFs of the shift adders under different precision modes since the utilization of the shift adder in different precision modes is low in prior research. Fig. 5(a) takes signed INT4/8 precision-reconfigurable situation as an example to explain the reason. In the INT4 mode, the least significant 4 bits are useless since in the INT4 mode there are only two PPs, the 4 bits are only useful when there are four PPs for shift-accumulation. The useless bits would increase to 12 for INT4/8/16 precision-reconfigurable situation. In the INT8 mode, there are many "0" bits in the sums of PPs generated by subtree since adder gating is used on the skipped adders to reduce energy consumption. These "0" bits cause low utilization of the shift adder. Because removing these bits does not change the result of the subshift adders in the INT8 mode.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:37 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE SUMMARY AND COMPARISON WITH OTHER DESIGNS

	TVLSI'23 [12]	JSSC'22 [13]	ISSCC'21-16.4 [6]	DAC'20 [14]	ISSCC'23-16.3 [15]	JSSC'24 [16]	This Work
Macro Type	Analog CIM	Analog CIM	Digital CIM	Digital CIM	Digital CIM	Digital CIM	Digital CIM
Technology	28nm	28nm	22nm	28nm	28nm	28nm	28nm
Supply Voltage (V)	0.7/0.9	0.7-0.9	0.72	0.6-1.1	0.469-0.9	0.6-0.9	0.9
Macro Size (kb)	4	384	64	64	Not reported	64	16
Peak Performance (GOPS)	26.8(INT4)	768(INT4) 213(INT8)	3300(INT4) 917(INT8)	0.21(INT8)	1640(INT4)	112.3-375(INT8) 89.8-300(BF16)	2048(INT4) 512(INT8) 128(INT16)
Energy Efficiency (TOPS/W)	19.7(INT4)	60.28(INT4) 15.02(INT8)	56.96(INT4) 15.81(INT8)	4.4(INT8)	74.68(INT4) 18.65(INT8) 4.67(FP16)	19.5(INT8) 14.04(BF16)	79.15 (INT4) 24.75 (INT8) 6.93 (INT16)

As shown in Fig. 5(b), the proposed PRSA is based on the insight that an adder with large bit-width can be divided into several adders with small bit-width by cutting down the carry connection. First, we determine the bit-width of the adder with the largest bit-width according to the largest bit-width of the result of different precision modes. For the signed INT4/8/16 situation, the largest bit-width is 56 since in the INT4 mode the adder tree outputs four 14-bit partial results. Second, we use multiple MUXs to control the bits of the adder with large bit-width to cut down the carry connection. Moreover, we rearrange the input of the shift adder under different precision modes. To further reduce the number of MUXs needed to arrange the inputs of the adders, we use an adder with large bit-width to accumulate the 3/2/1 groups of partial results under the INT 4/8/16 mode and set the bit-width to be 44 and used another 14-b adder to accumulate the other partial result for the INT4 mode. The 14-b adder is gated in other precision modes to reduce energy.

III. DESIGN IMPLEMENTATION AND EVALUATION

A. Design Implementation

After conducting research on the commonly used data precisions in DNN applications, we have selected to design the Booth macro supporting the precisions of the signed INT4/INT8/INT16 for both the input and weight with the proposed methodology. The layout is shown in Fig. 6(a). The proposed SRAM-CIM macro is designed in a standard 28-nm CMOS process. We designed the digital circuits in Verilog and then proceeded with the synthesis, autoplace, and routing (APR) using Synopsys Design Compiler. To provide an accurate assessment of the entire system, we conducted SPICE simulations of both the analog and digital parts. The power breakdown shown in Fig. 6(b) is the average power breakdown in the signed INT4, INT8, and INT16 modes. It is measured at a voltage of 0.9 V and a frequency of 500 MHz, at 25 °C. In the INT4 mode, all the bits of adders are useful, while in the INT8 and INT16 modes, many bits of adders are skipped. Using adder gating on the skipped useless bits of adders in the adder trees under the INT8 and INT16 modes, we reduced the energy greatly, as shown in Fig. 6(c).

B. Performance Evaluation

To evaluate the benefits and overhead of each technique of the proposed work, we have made a technique breakdown analysis for energy optimization of the proposed macro as shown in Fig. 6(d). We refer to the macro proposed in ISSCC'22-15.5 [5] to implement the baseline macro. We have also implemented variants of our macro. The variant Base + BP equips the baseline with the optimized PP generation method based on Booth. The variant Base + BP + RA further enhances the Base + BP by adding the precision-reconfigurable adder tree designed with the proposed methodology. The proposed macro finally uses the proposed PRSA. As shown in Fig. 6(d), the

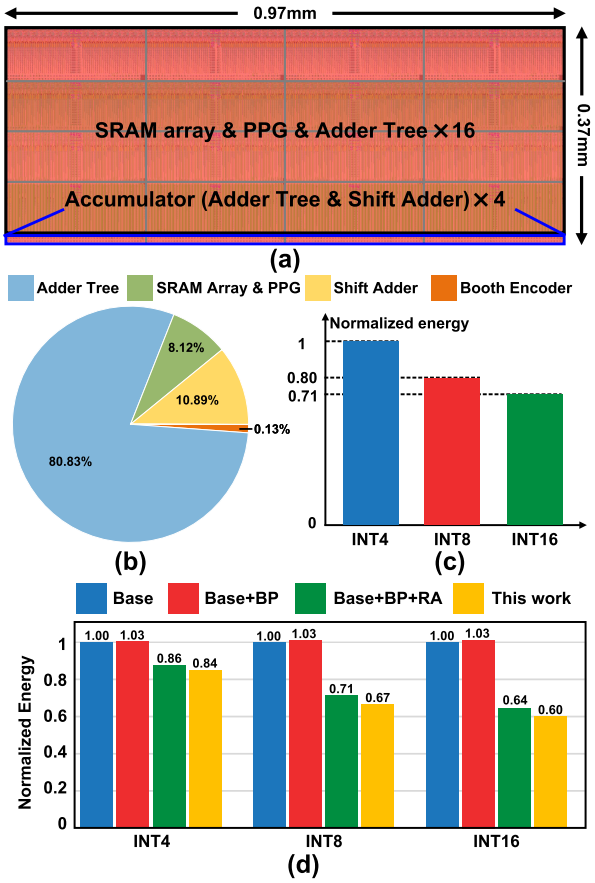


Fig. 6. (a) Layout of the proposed CIM macro. (b) Energy breakdown. (c) Energy of the proposed macro in different precision modes. (d) Technique breakdown analysis.

variant Base + BP consumes more power than baseline, and the energy consumption of Base + BP + RA has decreased significantly. Because joint optimization is performed for the BP and RA techniques. Base + BP merges the addition of Comp to all the stages of the adder tree by converting the LSB of each adder in all the stages of the adder tree into a full adder. As shown in Fig. 4, the Comp input for subtree1 is replaced by carry for higher precision modes. Therefore, the power overhead to achieve precision reconfiguration with the proposed precision-reconfigurable macro is very small, resulting in a significant improvement in energy consumption of Base + BP + RA compared with baseline.

Table II shows the postlayout simulation results of the proposed work and comparison to other designs. The compared works represent the two mainstream CIM macro architectures: analog CIM [12], [13] and digital CIM [6], [14], [15], [16]. For fair comparison, we scale the energy efficiency under the same supply voltage (0.9 V).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:37 UTC from IEEE Xplore. Restrictions apply.

The throughput and energy efficiency of the proposed macro are higher compared with previous works, primarily because the PP generation method based on Booth multiplication is used.

IV. Conclusion

This brief proposes a BCIM with modified Booth encoding and PP generation method for the CIM architecture and the methodology for designing precision-reconfigurable macros. The design achieves 2048 GOPS peak performance and 79.15 TOPS/W energy efficiency in the signed INT4 mode, which has both higher power efficiency and flexibility than many previous works.

References

[1] H. Hu et al., "A 40-nm SONOS digital CIM using simplified LUT multiplier and continuous sample-hold sense amplifier for AI edge inference," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 12, pp. 2044–2052, Dec. 2023.

[2] S. Yu, H. Jiang, S. Huang, X. Peng, and A. Lu, "Compute-in-memory chips for deep learning: Recent trends and prospects," *IEEE Circuits Syst. Mag.*, vol. 21, no. 3, pp. 31–56, 3rd Quart., 2021.

[3] W. Gul, M. Shams, and D. Al-Khalili, "FinFET 6T-SRAM all-digital compute-in-memory for artificial intelligence applications: An overview and analysis," *Micromachines*, vol. 14, no. 8, p. 1535, Jul. 2023.

[4] B. Yan et al., "A 1.041-Mb/mm^2 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28 nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2022, pp. 188–190.

[5] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2022, pp. 1–3.

[6] Y.-D. Chih et al., "An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2021, pp. 252–254.

[7] E.-J. Chang et al., "A 12-nm 0.62–1.61 mW ultra-low power digital CIM-based deep-learning system for end-to-end always-on vision," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Kyoto, Japan, Jun. 2023, pp. 1–2.

[8] F. Tu et al., "TranCIM: Full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," *IEEE J. Solid-State Circuits*, vol. 58, no. 6, pp. 1798–1809, Jun. 2023.

[9] Y. Fu, Y. Zhao, Q. Yu, C. Li, and Y. Lin, "2-in-1 accelerator: Enabling random precision switch for winning both adversarial robustness and efficiency," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, New York, NY, USA, Oct. 2021, pp. 225–237.

[10] H. Kim, T. Yoo, T. T. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[11] H. Mori et al., "A 4 nm 6163-TOPS/W/b 4790-TOPS/mm^2/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 132–134.

[12] Z. Tong, Y. Zhao, J. Zhang, Z. Lin, X. Lin, and X. Wu, "In-memory transposable multibit multiplication based on diagonal symmetry weight block," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 9, pp. 1454–1458, Sep. 2023.

[13] J. W. Su et al., "A 8-b-precision 6T SRAM computing-in-memory macro using segmented-bitline charge-sharing scheme for AI edge chips," *IEEE J. Solid-State Circuits*, vol. 58, no. 3, pp. 877–892, Mar. 2023.

[14] K. Lee, J. Jeong, S. Cheon, W. Choi, and J. Park, "Bit parallel 6T SRAM in-memory computing with reconfigurable bit-precision," in *Proc. 57th ACM/IEEE Des. Autom. Conf. (DAC)*, San Francisco, CA, USA, May 2020, pp. 1–6.

[15] J. Yue et al., "A 28 nm 16.9–300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 1–3.

[16] A. Guo et al., "A 28-nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, Sep. 2024.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:37 UTC from IEEE Xplore. Restrictions apply.